Python Programming - 2301CS404

Lab - 15 (1)

Roll No.: 315

Name: Vishva M. Bhimani

24010101026

01) Write a Program to create a class by name Students, and initialize attributes like name, age, and grade while creating an object.

In [6]:
```
class Student :
    def __init__(self,name,age,grade):
        self.name = name ,
        self.age = age ,
        self.grade = grade

s1 = Student("Vishva" ,18 , "A1")
print(s1.name)
print(s1.age)
print(s1.grade)
```

```
('Vishva',)
(18,)
A1
```

02) Create a class named Bank_Account with Account_No, User_Name, Email,Account_Type and Account_Balance data members. Also create a method GetAccountDetails() and DisplayAccountDetails(). Create main method to demonstrate the Bank_Account class.

In [11]:
```
class  Bank_Account:
    def __init__(self,User_Name,Email,Account_Type, Account_Balance):
```

```
        self.User_Name = User_Name ,
        self.Email = Email ,
        self.Account_Type = Account_Type ,
        self.Account_Balance = Account_Balance

    def DisplayAccountDetails(self):
        print("user_Name :",self.User_Name)
        print("Email is :" , self.Email)
        print("Account_Type is :",self.Account_Type)
        print("Account_Balance is :",self.Account_Balance)

s1 = Bank_Account("Vishva","abc@gmail.com" , "saving" , 4500)
s1.DisplayAccountDetails()
```

```
user_Name : ('Vishva',)
Email is : ('abc@gmail.com',)
Account_Type is : ('saving',)
Account_Balance is : 4500
```

03) WAP to create Circle class with area and perimeter function to find area and perimeter of circle.

In [12]:
```
class Circle:
    def __init__(self,r):
        self.r = r


    def area(self):
        return 3.14*self.r*self.r

    def perimeter(self):
        return 2*3.14*self.r

c1 = Circle(20)
a = c1.area()
b = c1.perimeter()
print("area is : ",a)
print("perimeter is :" ,b)
```

```
area is :  1256.0
perimeter is : 125.60000000000001
```

04) Create a class for employees that includes attributes such as name, age, salary, and methods to update and display employee information.

In [13]:
```
class employees:
    def __init__(self,name,age,salary):
        self.name = name,
        self.age = age,
        self.salary = salary

    def update(self,newname , newage , newsalary):
```

```
        self.name = newname ,
        self.age = newage,
        self.salary = newsalary

    def display(self):
        print("name is:",self.name)
        print("age is :" , self.age)
        print("salary is :",self.salary)

e1 = employees("vishva" , 18 , 45000)
e1.update("vishu" , 20 , 75000)
print("value is updated")
e1.display()
```

```
value is updated
name is: ('vishu',)
age is : (20,)
salary is : 75000
```

05) Create a bank account class with methods to deposit, withdraw, and check balance.

In [22]:
```
class Bank:
    def __init__(self,balance):
        self.balance = balance

    def Deposit(self,deposit):
        self.deposit = deposit
        self.balance += self.deposit


    def withdraw(self,w):
        self.w = w
        if (self.balance>self.w):
            self.balance -= self.w
        else:
            print("balance is insufficent")

    def display(self):
        print("balance is:",self.balance)

s1 = Bank(45000)
s1.Deposit(12000)
s1.display()
s1.withdraw(10000)
s1.display()
```

```
balance is: 57000
balance is: 47000
```

06) Create a class for managing inventory that includes attributes such as item name, price, quantity, and methods to add, remove, and update items.

In [35]:
```
class inventory:
    def __init__(self,name,price,quantity):
        self.n = name,
        self.p = price,
        self.q = quantity

    def Add(self,qty):
        print("name is :" , self.n)
        print("price is :" , self.p)
        print("quantity is :" , self.q)


    def remove(self,qty):

        if(qty>self.q):
            print("not is attribute")
        else:
           self.q -= qty
           print("remove sucessfully")


    def updated(self,newname,newprice,newquantity):
        self.n = newname,
        self.p = newprice,
        self.q = newquantity,


    def display(self):
        print("name is :" , self.n)
        print("price is :" , self.p)
        print("quantity is :" , self.q)


s1 = inventory("vishva" , 1200 , 3)
s1.Add(3)
s1.remove(3)
s1.updated("vishu" , 5000 , 5 )
s1.display()
```

```
name is : ('vishva',)
price is : (1200,)
quantity is : 3
remove sucessfully
name is : ('vishu',)
price is : (5000,)
quantity is : (5,)
```

07) Create a Class with instance attributes of your choice.

In [38]:
```
class new:
    def __init__(self,name,age):
```

```
        self.n = name,
        self.a =  age

    def display(self):
        print("name is :" , self.n)
        print("age is :" , self.a)

s1 = new("vishva", 12)
s2 = new("vishu" ,20)
print(s1.n)
print(s2.n)
print(s1.a)
print(s2.a)
```

```
('vishva',)
('vishu',)
12
20
```

08) Create one class student_kit

Within the student_kit class create one class attribute principal name (Mr ABC)

Create one attendance method and take input as number of days.

While creating student take input their name .

Create one certificate for each student by taking input of number of days present in class.

In [40]:
```
class student_kit:
    principal_name = "Mr.abc"
    def __init__(self,stu_name):
        self.n = stu_name

    def attendance(self,days):
        self.d = days

    def generate_certificate(self):
       print("\n--- Certificate ---")
       print("This is to certify that", self.n)
       print("has attended", self.d, "days of classes.")
       print("Principal:", student_kit.principal_name)

name = input("Enter Student Name: ")
days = int(input("Enter Number of Days Present: "))

s1 = student_kit(name)
s1.attendance(days)
s1.generate_certificate()
```

```
--- Certificate ---
This is to certify that vishva
has attended 45 days of classes.
Principal: Mr.abc
```

09) Define Time class with hour and minute as data member. Also define addition method to add two time objects.

In [41]:
```python
class Time:
    def __init__(self, h=0, m=0):
        self.h = h
        self.m = m

    def add_time(self, t2):
        h = self.h + t2.h
        m = self.m + t2.m

        while m >= 60:
            m = m - 60
            h = h + 1

        return Time(h, m)

    def display(self):
        print(self.h, "hour(s)", self.m, "minute(s)")



t1 = Time(2, 45)
t2 = Time(3, 30)

t3 = t1.add_time(t2)

t3.display()
```

```
6 hour(s) 15 minute(s)
```

In [ ]:
```python

```